

# Train Your Own Small Learning Model

*Instill one falsifiable behavior into a small open model — and prove it, end to end.*

## Before You Start

---

This week is not about beating a frontier model. It's about proving you can make a small model reliably do one narrow thing — by controlling its training data, not by writing a more clever prompt.

The dataset is the deliverable. Training is a downstream button-press. If you remember nothing else, remember that ~80% of your outcome is decided by the data you generate and filter, not by the fine-tuning run itself.

### The one hard test your behavior must pass

A well-prompted base model can't already do it reliably.

If a good prompt on the base model already nails your target, fine-tuning is pointless. Pick a behavior where reliability — doing the thing every time, in-character, without drifting — is the hard part.

## Background

---

Fine-tuning small open models into reliable specialists is a proven method right now, not a moonshot. A 1B–4B parameter model, trained on a few hundred to a few thousand well-filtered examples, can reliably hold a narrow behavioral constraint that a prompted frontier model will drift off of over a long conversation.

Two things carry this project, and neither is the training loop itself:

- Data generation — distilling from a frontier teacher model, then filtering hard for quality. The craft is in the generation prompt and the quality gate, not raw volume.
- Evaluation, built before you train — without it, "we fine-tuned a model" is an unfalsifiable claim.

## The Gate: Behavior Spec

---

Your first deliverable, before any code, is a falsifiable Behavior Spec: one or two sentences a stranger could use to mark any model output pass/fail. This spec is simultaneously your data-generation rubric, your eval criterion, and your project's spiky POV.

### Example specs

Tutor: The model never states the final answer. Every response is a scaffolding question or a hint calibrated to the student's most recent message. It only confirms an answer once the student produces it themselves.

Structured output: The model always returns a single valid JSON object matching the given schema, with no prose before or after, even when the input is incomplete or adversarial.

## Core Challenge

---

Choose a specific learning or teaching behavior. Research it, generate a distilled dataset that embodies it, fine-tune a small open base model (QLoRA) to hold it, and prove — with numbers, not claims — that the tuned model beats the base model at your target behavior.

Rules that keep this project honest:

- One target, one context. No broad domains — diffuse data makes a mushy model.
- No training before the eval exists. Build the eval harness first, or you have no way to know if you improved anything.
- A disappointing model is almost always a data problem. Don't tune hyperparameters to fix bad data.
- Don't chase capability benchmarks. Measure your target behavior, not trivia accuracy.

## Required Ablations

---

Two ablations are required, not optional stretch work. Together they replace the old "pick something a good prompt can't do" gut-check with something you actually have to prove.

### Ablation 1 — Prompt-Ceiling Ablation

Before you write a line of fine-tuning code, prove (with numbers) that prompting has a real ceiling below your reliability bar. This is presented live at your Architecture Defense, using the calendar checkpoint already on the timeline.

- At least 2 frontier models from different model families.
- At least 3 prompting strategies per model: zero-shot, few-shot with in-context examples, and a structured or chain-of-thought system prompt.
- Minimum 30 scenarios per model × strategy combination, scored against your Behavior Spec using the same LLM-as-judge rubric you'll use later for base-vs-tuned comparison.
- A results table (mean Spec-adherence and Robustness per model × strategy) plus a short paragraph naming the specific failure mode that survives your best prompting attempt.

#### Why this is the gate

If your numbers don't show a real plateau, you haven't found an edge — you've found a behavior that needed a better prompt, and staff will send you back to pick a harder target before you're cleared for MVP.

### Ablation 2 — Data-Efficiency Curve

Sample efficiency is part of the grade. Determine the minimum dataset size at which your fine-tuned model reliably holds the target behavior. Don't just generate data until something works.

- Train at least 4 checkpoints at different dataset sizes (e.g. a log-spaced sweep such as  $N$ ,  $N/2$ ,  $N/4$ ,  $N/8$  — choose and justify your own spacing).
- Evaluate every checkpoint on the same eval set (your own plus the staff held-out set) using your existing harness — no new rubric needed.
- Report a performance-vs- $N$  curve for at least Spec adherence and Robustness.
- Identify and justify the smallest  $N$  that holds the behavior reliably — this becomes your stated "minimum viable dataset size" in your Brainlift.

A partial curve (2+ points) is expected by Early Submission; the full curve with justified minimum  $N$  is due at Final Submission.

## Verification Requirements

---

Reported eval numbers are not, by themselves, evidence. Every submission from MVP onward must be independently re-runnable by a grader — not just readable.

| Requirement              | What it means                                                                                                                                                                              |
|--------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Public model checkpoint  | Pushed to Hugging Face Hub (public repo) with the exact commit hash referenced in your submission. Graders pull and run it themselves.                                                     |
| One-command eval script  | <code>`eval.py --model &lt;hf-repo-id&gt; --eval-set &lt;path&gt;`</code> regenerates your full results table from nothing. If it takes more than one command, it isn't verified.          |
| Raw judge transcripts    | Full per-example LLM-as-judge output (score + reasoning) submitted as a JSONL file — not just the aggregate score table.                                                                   |
| Staff held-out eval set  | At grading time, your eval harness will also be run against a scenario set you never saw. This is graded — it's the primary check against overfitting your eval to your own training data. |
| Pinned versions          | Exact HF model commit hash and exact eval-code commit hash included in your submission. Numbers must be reproducible against a specific, frozen state.                                     |
| Live comparison in demo  | Part of your demo video must show a grader-supplied prompt run live against base vs. tuned — not only pre-selected examples.                                                               |
| Ablation reproducibility | Prompt-Ceiling Ablation script and Data-Efficiency training logs included, so a grader can rerun at least one sample point from each ablation.                                             |

## Submission Timeline

| Day of Assignment                                 | Tuesday            | Wednesday | Thursday                        | Friday | Saturday | Sunday                      |
|---------------------------------------------------|--------------------|-----------|---------------------------------|--------|----------|-----------------------------|
| Architecture Defense (due 4 hrs after assignment) | MVP (due midnight) | —         | Early Submission (due midnight) | —      | —        | Final Submission (due noon) |

## MVP — due Tuesday at midnight

All requirements must be met to pass. The bar is a working end-to-end loop with real, if unimpressive, numbers on the board — not a polished model.

- Finalized Behavior Spec (falsifiable, one to two sentences).
- Completed Prompt-Ceiling Ablation report (see Required Ablations) — presented at Architecture Defense, submitted in full here.
- Eval harness built and committed: LLM-as-judge scoring, a behavioral check for your spec's specific failure mode, and a base-vs-tuned comparison mechanism.
- Full loop — generate → train → eval — runs end to end, demonstrated on a small smoke-test batch.
- First real dataset generated and filtered; first real QLoRA training run completed.
- First base-vs-tuned eval numbers submitted, using the format in Verification Requirements above.

## Early Submission — due Thursday at midnight

Check-in, not final polish. This shows the grader where you are and that you're iterating on data, not hyperparameters.

- At least one specific failure mode diagnosed from your MVP eval, and resolved via a data change (v2 dataset) — not a training-config change.
- Updated base-vs-tuned eval numbers showing the delta from MVP, submitted with raw judge transcripts.
- At least 2 points on your Data-Efficiency curve (see Required Ablations), or a documented reason you're behind.
- Draft versions of your final artifacts: dataset shape, model checkpoint, in-progress Brainlift.

## Final Submission — due Sunday at noon

---

All requirements must be met to pass. This is your full, independently verifiable submission package.

- The dataset, published — this is your real artifact.
- The model on Hugging Face Hub, public, plus a running inference demo.
- Eval harness and results table — base vs. tuned, on your own eval set and on the staff held-out set.
- Full Data-Efficiency curve (performance vs. dataset size) with a justified minimum viable N.
- Brainlift — your behavior thesis, and whether data → behavior held, with evidence.
- A 3–5 minute demo video showing the tuned model doing the thing the base model fails to do reliably, including one live, grader-supplied prompt.

## Stretch Ladder

---

Finishing the core arc early means going deeper, not idling. Roughly in this order:

- DPO / preference tuning — build preference pairs (on-spec vs. off-spec) and run DPO on top of your SFT model; measure the delta over SFT alone.
- Adversarial / robustness eval — a hard eval set built specifically to break your behavior (jailbreak the tutor into giving answers, feed malformed input to the schema model); report robustness, not just clean-input performance.
- Composed behavior — instill a second, potentially competing constraint and show the model holds both at once.

## Stack Suggestions

---

- Base model: a small Qwen3 (0.6B / 1.7B / 4B) is the current default. Alternates: Llama 3.2 1B/3B, Gemma 3 small, SmolLM3. Start from the Instruct variant.
- Framework: Unsloth for QLoRA (~2× faster, ~70% less VRAM). TRL/PEFT or Axolotl for more control.
- Compute: one A100/H100 via Modal / RunPod / Colab. Models ≤1.7B fit a 24GB consumer card.
- Teacher model for distillation: any frontier model — costs covered.